Task 5: Silverbox oscillator

Neural acceleration modeling with a continuous RK4 free run

A fully connected PyTorch network predicts acceleration from position, velocity, and applied voltage. Integrating its state equation produces the output waveform. Three architectures and three training settings per architecture were compared, alongside the original tutorial configuration.

Selected result

Measurement	Value
Hidden-layer widths	64-64
Trainable parameters	4481
Full free-run training RMSE	0.00291446 V
Full free-run test RMSE	0.01149832 V
Zero-prediction test RMSE	0.06049186 V

Data preparation

The source is SNLS80mV.mat: channel V1 is input voltage u and channel V2 is output voltage y. The slice [42650:46746] contains 4,096 samples at 610.35 Hz. As required, the mean of each signal on the full excerpt is subtracted. The first 3,072 samples form the training record; the last 1,024 form the test record.

Physical model and state equation

$$y'' + a\,y' + b\,y + c\,y^3 = g\,u(t)$$

$$y' = v;\; v' = \alpha_\theta(y, v, u)$$

The Duffing equation motivates the acceleration model. The network represents the force; a, b, c, and g are not fitted as separate coefficients here. The state is [y, v]. Classical RK4 advances it with dt = 1/610.35 s, linearly interpolating the known input voltage between adjacent samples.

Training and model comparison

Every network has three inputs ordered [y, v, u], Tanh after each hidden linear layer, and one linear acceleration output. Training-only RMS scales are approximately 0.05302 V for position, 22.1815 V/s for finite-difference velocity, 0.02196 V for input, and 9,886.33 V/s^2 for acceleration. Physical states remain unscaled inside RK4.

Each update samples windows entirely within training data. Starting position is measured training y; starting velocity is (y[1]-y[0])/dt within that window. Later states are predicted, without measured-position corrections. The loss is mean squared position error averaged across time and batch. Backpropagation passes through RK4. Weight initialization uses seed 0; window sampling uses a separate generator with seed 1 for comparisons independent of network width. The original tutorial run also appears in the table; it uses the global sampling sequence after seed-0 weight initialization.

Training settings tested for every architecture

Setting	Optimizer	Rate	Updates	Window	Batch
1	Adam	0.01	3000	64	1
2	Adam	0.003	2000	64	8
3	AdamW	0.003	3000	128	8

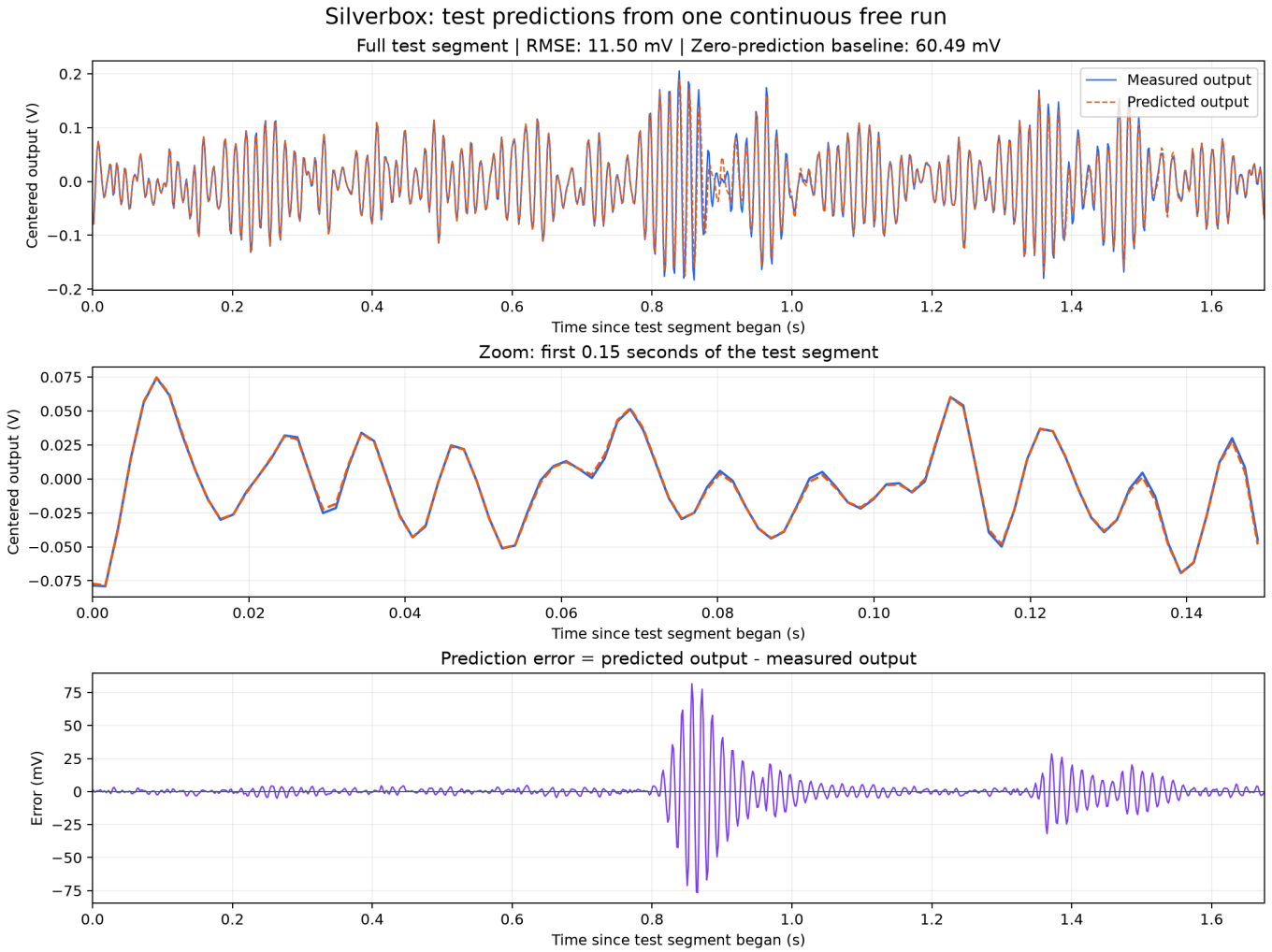
AdamW uses weight decay 0.0001. Settings vary several factors together; this is a small search rather than an isolated causal study.

Architecture	Setting	Params	Train mV	Test mV	Time (s)
32-32	1	1217	7.506	19.611	41.3
32-32	2	1217	5.021	16.788	33.4
32-32	3	1217	2.659	14.907	99.9
64-64	1	4481	16.536	20.060	49.7
64-64	2	4481	4.751	16.339	36.8
64-64	3	4481	2.914	11.498	114.3
32-32-32	1	2273	15.982	19.818	50.2
32-32-32	2	2273	6.400	21.123	38.9
32-32-32	3	2273	4.067	16.964	117.7
32-32	Original	1217	10.259	11.648	46.7

The selected version is **64x64_setting3**: AdamW, learning rate 0.003, 3000 updates, 128-sample windows, and batch size 8. It gives the lowest test free-run RMSE in this comparison, as required by the assignment. Test outputs enter scoring and model selection after fitting; they never enter the gradient calculation.

Continuous test prediction

After weights are fixed, the model starts at excerpt sample 0 with measured position and forward finite-difference velocity. It integrates all 4,096 samples in one run. The state is carried across the training/test boundary. Measured test y is used only to calculate error and draw this comparison after prediction.



Test RMSE is 11.498 mV, a 81.0% reduction from the zero-prediction baseline. Maximum absolute test error is 81.645 mV; mean signed error is 0.093 mV. The lower panel displays individual signed errors; RMSE summarizes their squared magnitudes.

Code and reproducibility

Run the comparison: `python src/experiments.py`
Evaluate saved weights: `python src/test.py --no-show`

Saved artifacts include the experiment CSV/JSON tables, per-run loss histories and checkpoints, selected weights, and full measured/predicted traces. The selected checkpoint was reloaded and its RMSE reproduced independently.

Repository: https://github.com/WhoEvenAm/Physics-Informed-Neural-Network-for-the-Silverbox-oscillator

Source: Homework 1, Task 5, pages 5-6; Silverbox SNLS80mV.mat benchmark record (nonlinearbenchmark.org).